Image Analysis: Convolutional Neural Networks

MSIS 521

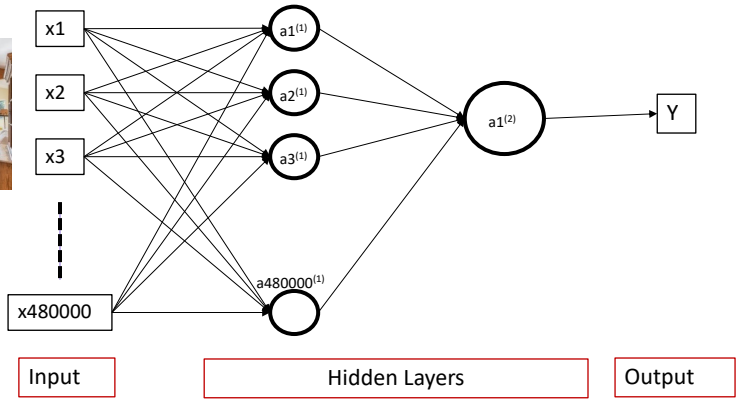
Professor Hossein Ghasemkhani

FOSTER SCHOOL OF BUSINESS
UNIVERSITY OF WASHINGTON

1

A Simple Deep NN model

400 X 400 X 3



Number of Parameters

> 480000*480000 + 480000 +1 = approximately 230 Billion!!!

> TOO MANY PARAMETERS!!!

FOSTER SCHOOL OF BUSINESS

2

2

How Convolutional Layers Help?

- Inspired by the neurophysiological experiments conducted by Hubel and Wiesel 1962

> Filter $\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$

Input Image

Convoluted Image

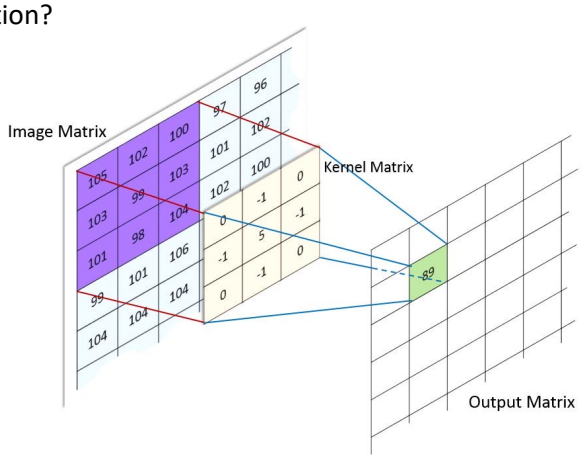
FOSTER SCHOOL OF BUSINESS

3

3

Convolution

- What is Convolution?



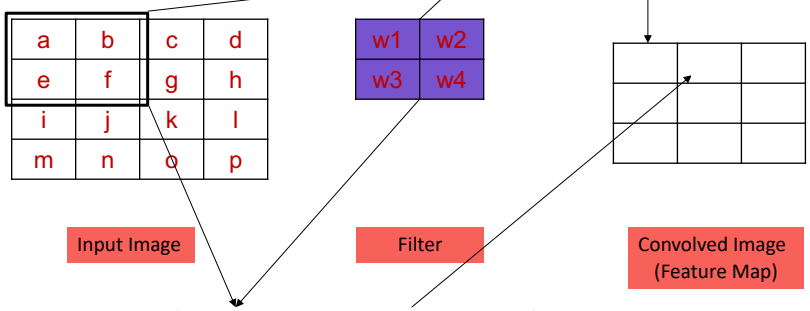
FOSTER SCHOOL OF BUSINESS

4

4

Convolution

$$h1 = f(a * w1 + b * w2 + e * w3 + f * w4)$$



$$h2 = f(b * w1 + c * w2 + f * w3 + g * w4)$$

- Number of Parameters for one feature map = 4 (using 1 filter)
- Size of filter determine size of convoluted image

FOSTER SCHOOL OF BUSINESS

5

5

Convolution

Kernel Matrix

105	102	100	97	96
103	99	103	101	102
101	98	104	102	100
99	101	106	104	99
104	104	104	100	98

Image Matrix

0	-1	0
-1	5	-1
0	-1	0

	89				

Output Matrix

$$105 * 0 + 102 * -1 + 100 * 0 + 103 * -1 + 99 * 5 + 103 * -1 + 101 * 0 + 98 * -1 + 104 * 0 = 89$$

FOSTER SCHOOL OF BUSINESS

6

6

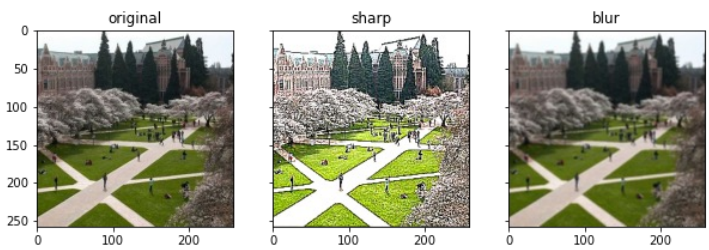
Blur or Sharpen

- Sharpen the image

$$\begin{pmatrix} -1 & -1 & -1 \\ -1 & 10 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

- Blur the Image

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} / 9$$



FOSTER SCHOOL OF BUSINESS

7

Python

```
from PIL import ImageFilter

sharp = ImageFilter.Kernel((3, 3), (-1, -1, -1, -1, 10, -1, -1, -1, -1), 1, 0)
im_sharp = im_na.filter(sharp)

blur = ImageFilter.Kernel((3, 3), (1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9), 1, 0)
im_blur = im_na.filter(blur)
```

FOSTER SCHOOL OF BUSINESS

8

8

Find Edge

- Laplacian kernel

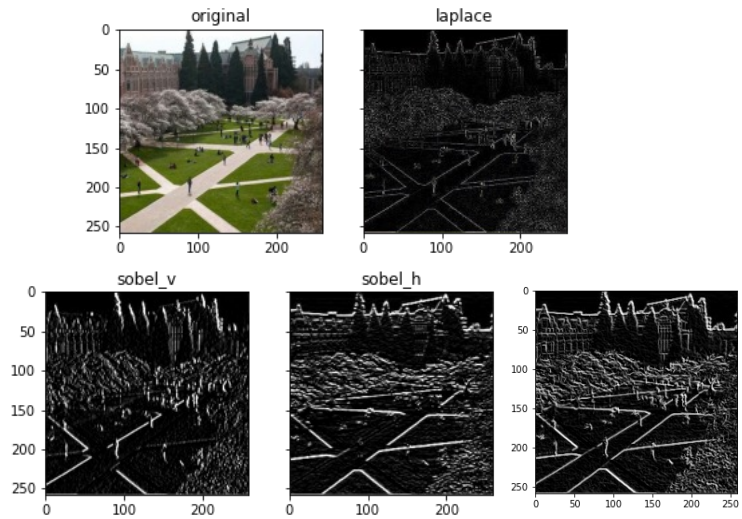
$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

- Sobel-v kernel

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

- Sobel-h kernel

$$\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$



FOSTER SCHOOL OF BUSINESS

9

9

Python

- Laplacian

```
laplace = ImageFilter.Kernel((3, 3), (0, 1, 0, 1, -4, 1, 0, 1, 0), 1, 0)

im_l = im_na.filter(laplace)
```

- Sobel

```
sobel_v = ImageFilter.Kernel((3, 3), (-1, 0, 1, -2, 0, 2, -1, 0, 1), 1, 0)
sobel_h = ImageFilter.Kernel((3, 3), (-1, -2, -1, 0, 0, 0, 1, 2, 1), 1, 0)

im_s_v = im_gray.filter(sobel_v)
im_s_h = im_gray.filter(sobel_h)
```

FOSTER SCHOOL OF BUSINESS

10

10

Convolution Layers

- The core layers in Convolutional NN (CNN)
- Kernel of unknown parameters rather than given values

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \qquad \begin{pmatrix} \beta_{11} & \beta_{12} & \beta_{13} \\ \beta_{21} & \beta_{22} & \beta_{23} \\ \beta_{31} & \beta_{32} & \beta_{33} \end{pmatrix}$$

FOSTER SCHOOL OF BUSINESS

11

11

Pooling

- Pooling is a common technique in deep learning for images to **downsample** an image while preserving important information.
- Max pooling: reports the maximum output within a rectangular neighborhood.
- Average pooling: reports the average output of a rectangular neighborhood.

1	3	5	3
4	2	3	1
3	1	1	3
0	1	0	4

Input Matrix

MaxPool with 2X2 filter
with stride of 2

4	5
3	4

Output Matrix

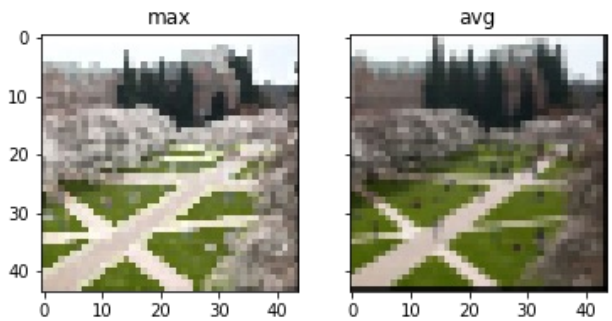
FOSTER SCHOOL OF BUSINESS

12

12

Python

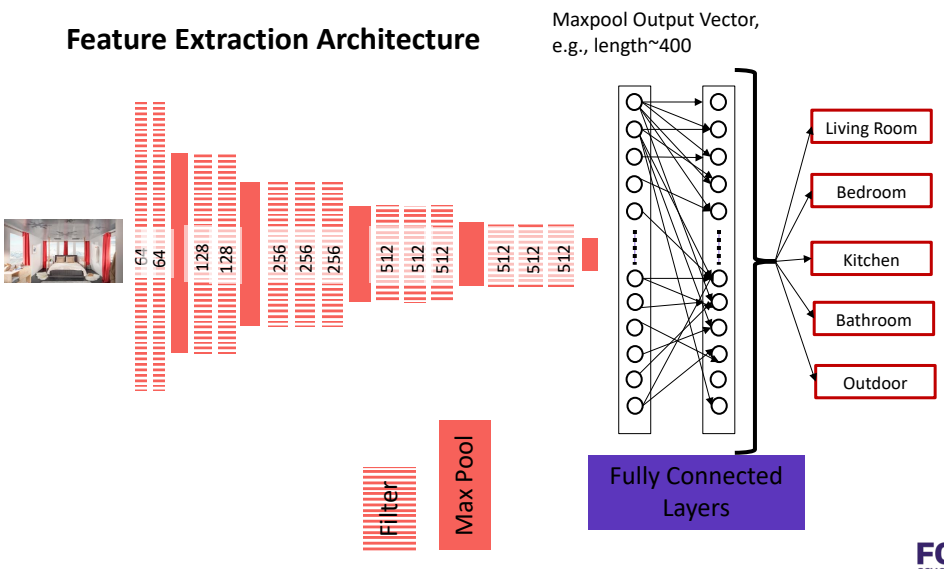
```python
from skimage.measure import block_reduce

im_max_pool = block_reduce(im_m, block_size = (6, 6, 1), func = np.max)
im_avg_pool = block_reduce(im_m, block_size = (6, 6, 1), func = np.mean).astype(uint8)
```



13

13

Convolutional Neural Network



14

14

CNN Example: Image Recognition

- Example by https://www.tensorflow.org/tutorials/images/cnn

- Import TensorFlow

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

- Download and prepare the CIFAR10 dataset

```
(train_images, train_labels), (test_images, test_labels) =   datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
```

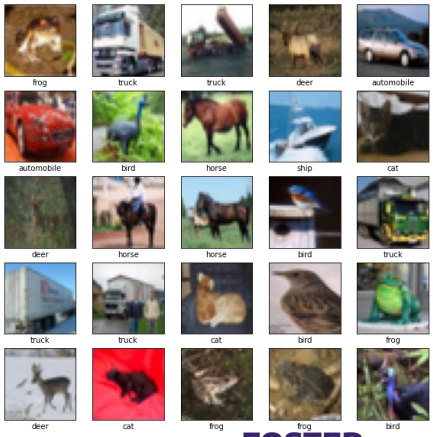
15

15

CNN Example: Image Recognition

- Verify the data

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



16

16

CNN Example: Image Recognition

- Create the convolutional base

```
model = models.Sequential()
model.add(layers.Input(shape=(32, 32, 3)))
model.add(layers.Conv2D(32, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.summary()
```

- Add Dense layers on top

```
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))

model.summary()
```

FOSTER SCHOOL OF BUSINESS

17

17

CNN Example: Image Recognition

- Compile and train the model

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

FOSTER SCHOOL OF BUSINESS

18

18

CNN Example: Image Recognition

- Evaluate the model

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')

test_loss, test_acc = model.evaluate(test_images,  test_labels, verbose=2)

print(test_acc)
```

FOSTER SCHOOL OF BUSINESS

19

19